Algorithms

Graph Searching Techniques
Topological Sorting

Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

Graph Searching

- Given: a graph G = (V, E), directed or undirected
- Goal: methodically explore every vertex and every edge
- Ultimately: build a tree on the graph
  - Pick a vertex as the root
  - Choose certain edges to produce a tree
  - Note: might also build a *forest* if graph is not connected

- There are two standard graph traversal techniques:
  - Breadth-First Search (BFS)
  - Depth-First Search (DFS)

Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

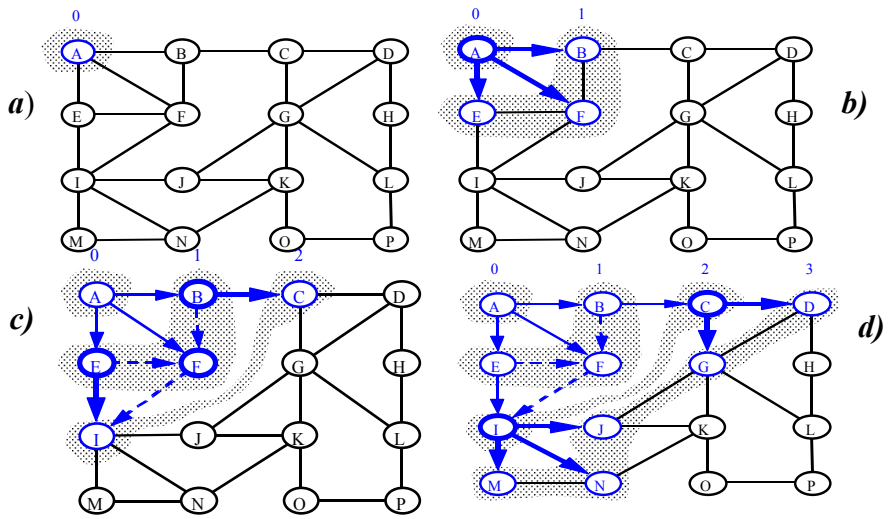
Breadth-First Search

- "Explore" a graph, turning it into a tree
  - One vertex at a time
  - Expand frontier of explored vertices across the *breadth* of the frontier
- Builds a tree over the graph
  - Pick a *source vertex* to be the root
  - Find ("discover") its children, then their children, etc.

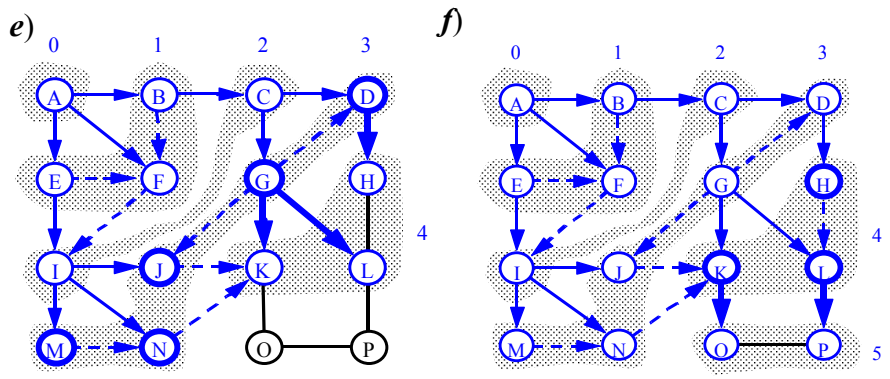
Breadth-First Search

- Again will associate vertex "colors" to guide the algorithm
  - White vertices have not been discovered
    - All vertices start out white
  - Grey vertices are discovered but not fully explored
    - They may be adjacent to white vertices
  - Black vertices are discovered and fully explored
    - They are adjacent only to black and grey vertices
- Explore vertices by scanning adjacency list of grey vertices

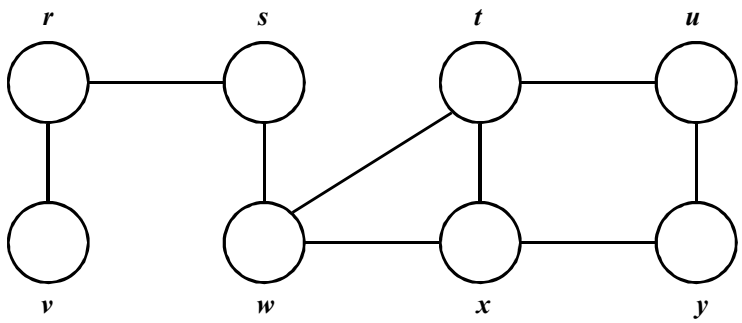
BFS - A Graphical Representation



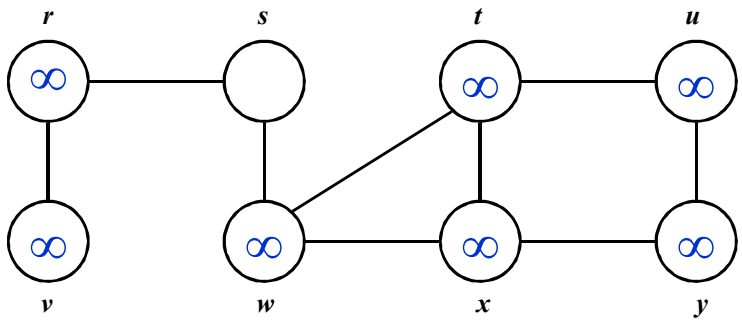
BFS - A Graphical Representation



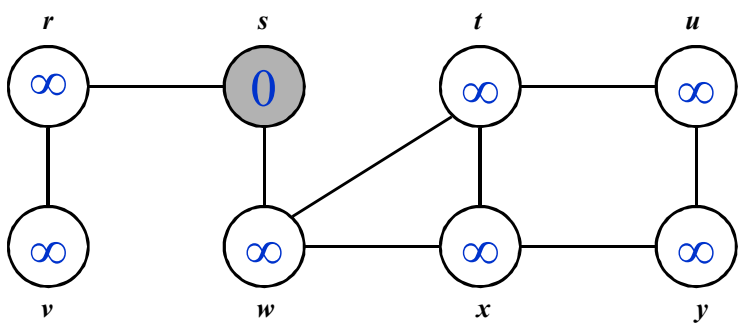
Breadth-First Search: Example



Breadth-First Search: Example

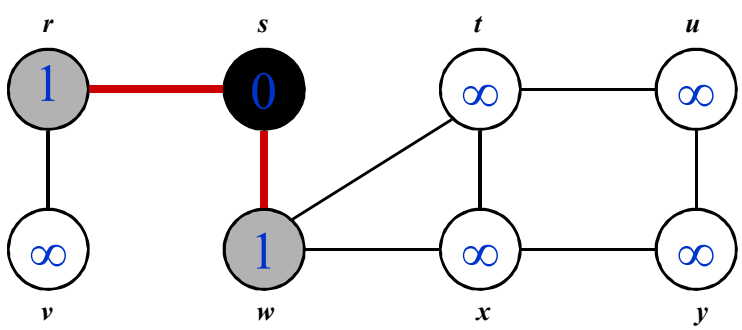


Breadth-First Search: Example



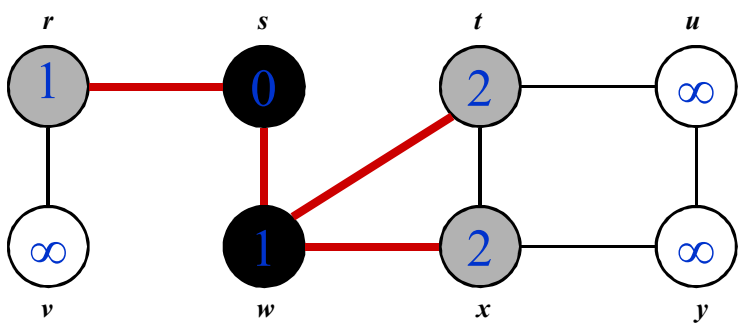
Q: | *s* |

Breadth-First Search: Example



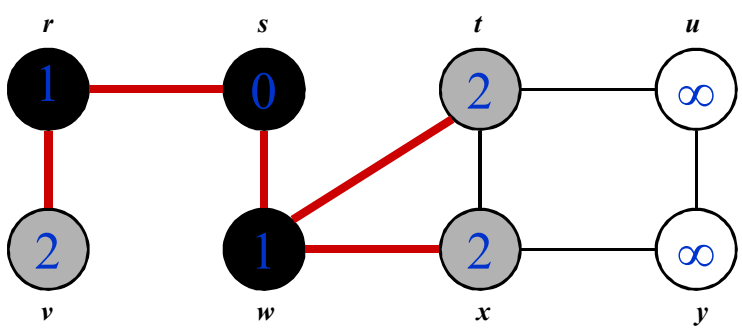
Q: | *w* | *r* |

Breadth-First Search: Example



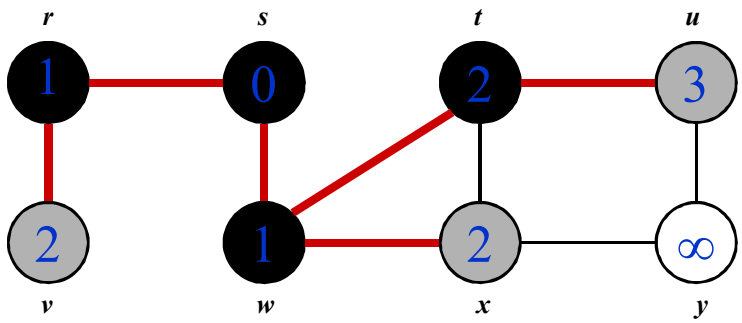
Q: | *r* | *t* | *x* |

Breadth-First Search: Example



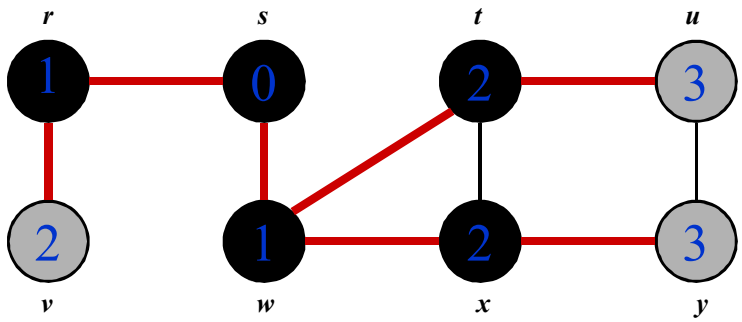
Q: | *t* | *x* | *v* |

Breadth-First Search: Example



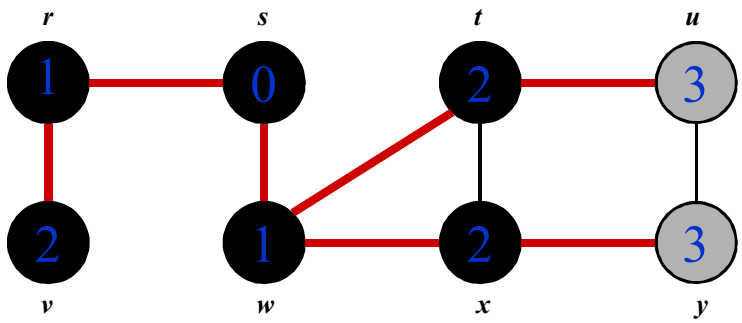
Q: | *x* | *v* | *u* |

Breadth-First Search: Example



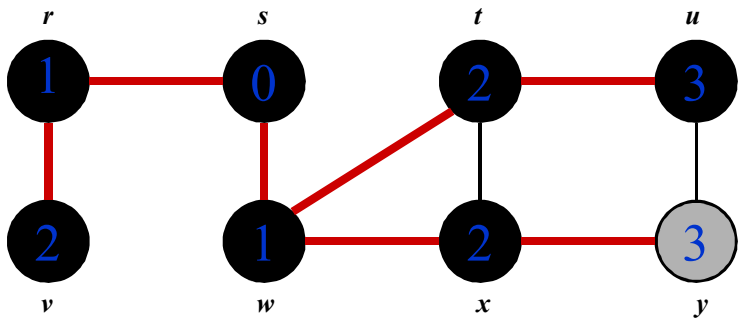
Q: | *v* | *u* | *y* |

Breadth-First Search: Example



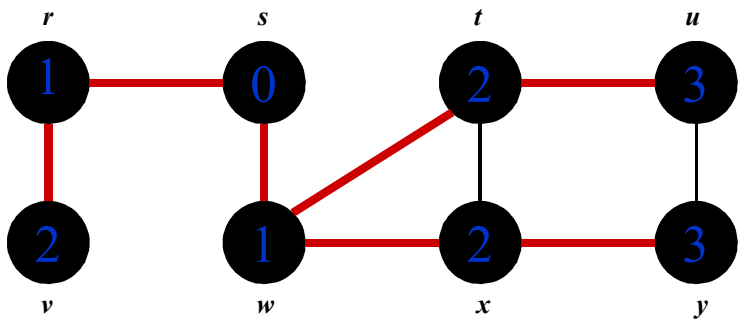
Q: | *u* | *y* |

Breadth-First Search: Example



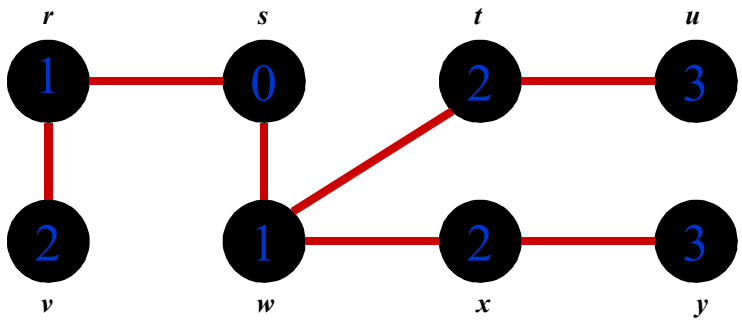
Q: | *y* |

Breadth-First Search: Example



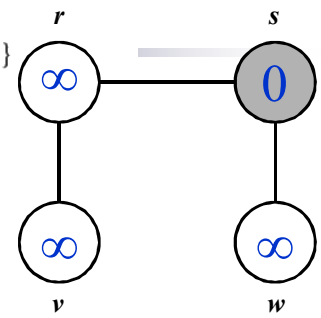
Q: Ø

Breadth-First Search: Example



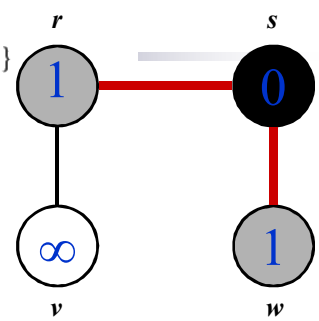
Output: BFS Spanning Tree

```
BFS(G, s)
for each vertex u ∈ V[G] − {s}
    do color[u] ← WHITE
       d[u] ← ∞
       π[u] ← NIL
color[s] ← GRAY
d[s] ← 0
π[s] ← NIL
Q ← ∅
ENQUEUE(Q, s)
while Q ≠ ∅
    do u ← DEQUEUE(Q)
       for each v ∈ Adj[u]
           do if color[v] = WHITE
                 then color[v] ← GRAY
                      d[v] ← d[u] + 1
                      π[v] ← u
                      ENQUEUE(Q, v)
       color[u] ← BLACK
```



Q: | s |

```
BFS(G, s)
for each vertex u ∈ V[G] − {s}
    do color[u] ← WHITE
       d[u] ← ∞
       π[u] ← NIL
color[s] ← GRAY
d[s] ← 0
π[s] ← NIL
Q ← ∅
ENQUEUE(Q, s)
while Q ≠ ∅
    do u ← DEQUEUE(Q)
       for each v ∈ Adj[u]
           do if color[v] = WHITE
                 then color[v] ← GRAY
                      d[v] ← d[u] + 1
                      π[v] ← u
                      ENQUEUE(Q, v)
       color[u] ← BLACK
```



Q: | w | r |

BFS: The Code Again

```
BFS(G, s) {
    initialize vertices;
    Q = {s};
    while (Q not empty) {
        u = RemoveTop(Q);
        for each v ∈ u->adj {
            if (v->color == WHITE)
                v->color = GREY;
                v->d = u->d + 1;
                v->p = u;
                Enqueue(Q, v);
        }
        u->color = BLACK;
    }
}
```

Touch every vertex: O(V)

u = every vertex, but only once (Why?)

So v = every vertex that appears in some other vert's adjacency list

What will be the running time?

Total running time:
$O(V + \Sigma(\text{degree}(v))) = O(V+E)$

BFS: The Code Again

```
BFS(G, s) {
    initialize vertices;
    Q = {s};
    while (Q not empty) {
        u = RemoveTop(Q);
        for each v ∈ u->adj {
            if (v->color == WHITE)
                v->color = GREY;
                v->d = u->d + 1;
                v->p = u;
                Enqueue(Q, v);
        }
        u->color = BLACK;
    }
}
```

What will be the storage cost in addition to storing the tree?

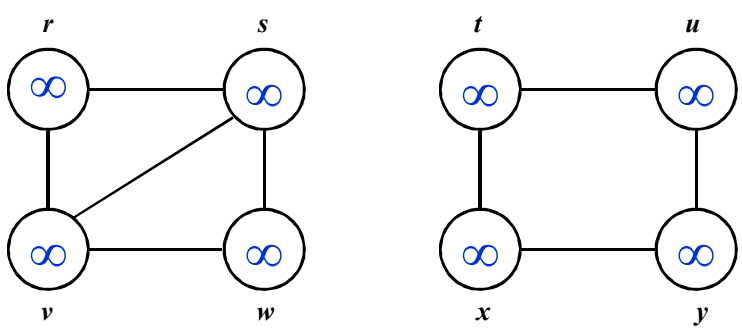
Total space used:
$O(V + \Sigma(\text{degree}(v))) = O(V + E)$

Breadth-First Search : Disconnected Graph

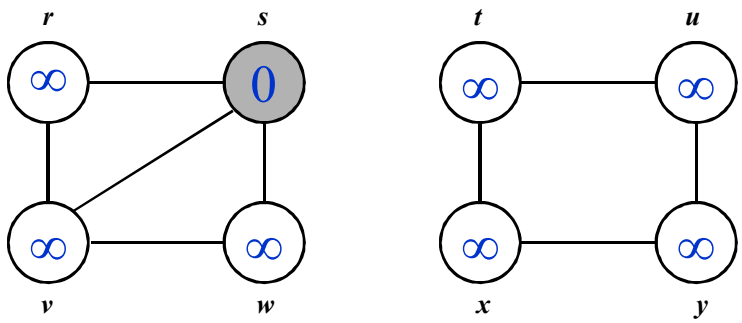
```
BFS(G)
  for each vertex u ∈ G->V
      u->color = WHITE;
      u->d = ∝
      u->π = NIL

  for each vertex u ∈ G->V
      if (u->color == WHITE)
          BFS_Visit(u);
```

```
BFS_Visit(u)
  u->color = GREY;
  u->d = 0
  u->π = NIL
  Q = ϕ;
  Enqueue(Q, u)
  while Q ≠ ϕ
      u = Dequeue(Q)
      for each v ∈ u->Adj[ ]
          if (v->color == WHITE)
              v->color = GREY;
              v->d = u->d + 1
              v->π = u
              Enqueue(Q, v)
      u->color = BLACK;
```


Breadth-First Search : Disconnected Graph

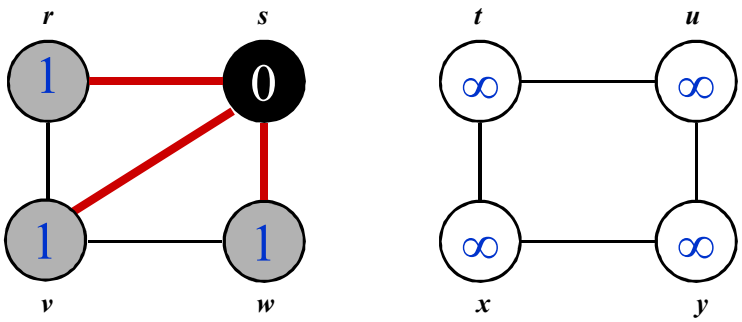


Breadth-First Search : Disconnected Graph



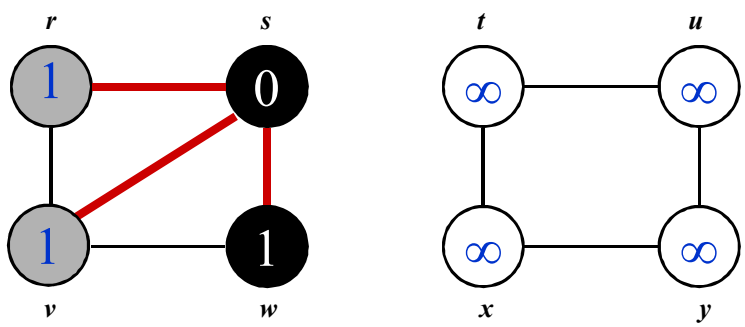
Q: | *s* |

Breadth-First Search : Disconnected Graph



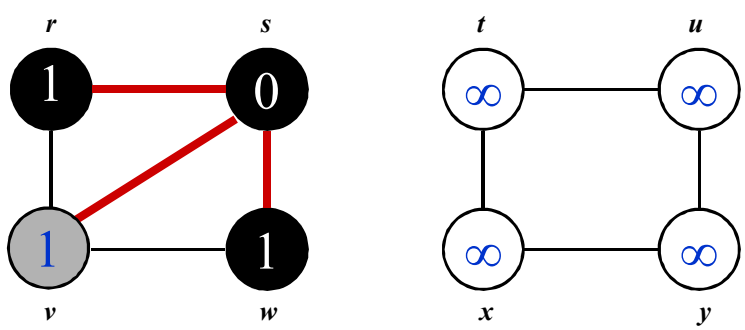
Q: | *w* | *r* | *v* |

Breadth-First Search : Disconnected Graph



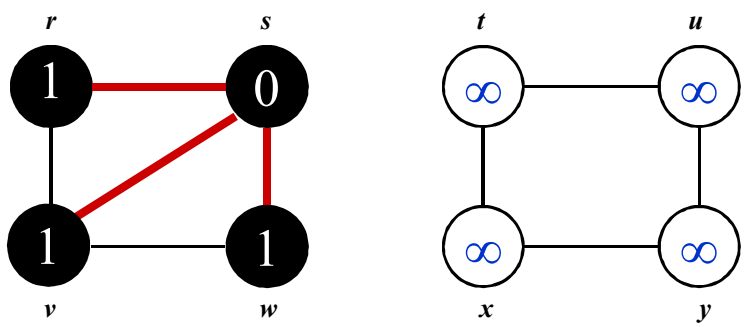
Q: | *r* | *v* |

Breadth-First Search : Disconnected Graph



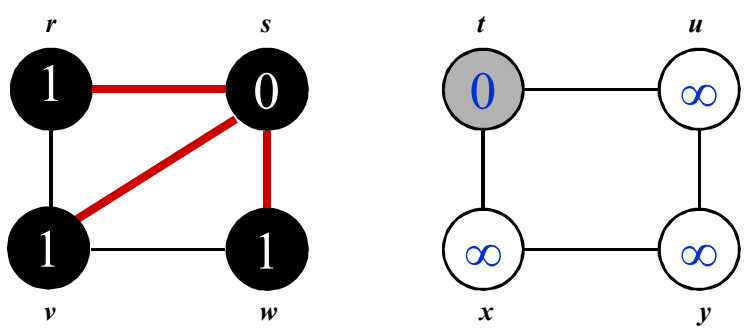
Q: | *v* |

Breadth-First Search : Disconnected Graph



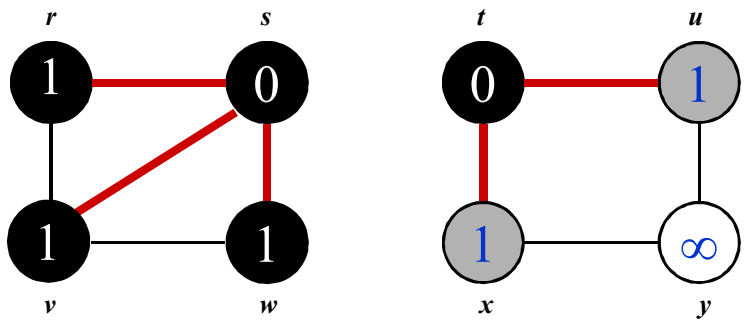
Q: Ø

Breadth-First Search : Disconnected Graph



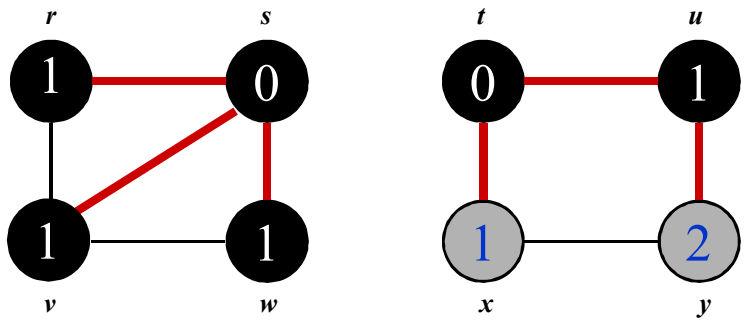
Q: $\boxed{t}$

Breadth-First Search : Disconnected Graph



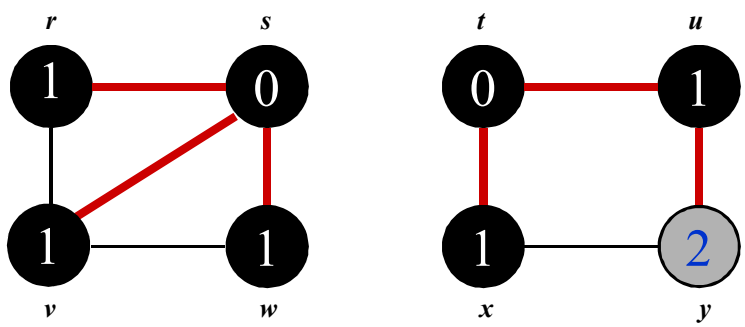
Q: | *u* | *x* |

Breadth-First Search : Disconnected Graph



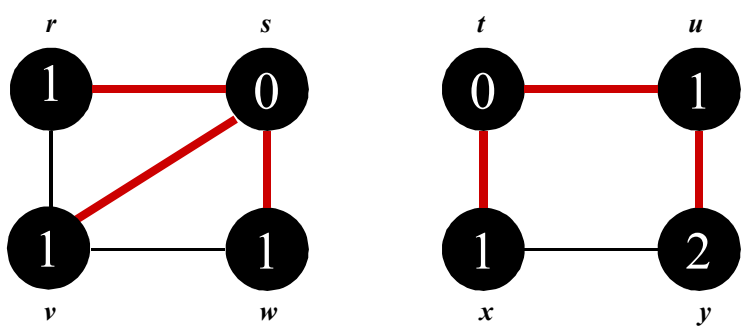
Q: | *x* | *y* |

Breadth-First Search : Disconnected Graph



Q: $\boxed{y}$

Breadth-First Search : Disconnected Graph



Q: Ø

Breadth-First Search: Properties

- BFS calculates the *shortest-path distance* to the source node
  - Shortest-path distance $\delta(s, v)$ = minimum number of edges from s to v, or ∞ if v not reachable from s
- BFS builds *breadth-first spanning tree* (*forest*), in which paths to root(s) represent shortest paths in G
  - Thus can use BFS to calculate shortest path from one vertex to another in $O(V + E)$ time in an unweighted graph

Depth-First Search

- *Depth-first search* is another strategy for exploring a graph
  - Explore "deeper" in the graph whenever possible
  - Edges are explored out of the most recently discovered vertex v that still has unexplored edges
  - When all of v's edges have been explored, backtrack to the vertex from which v was discovered
- Vertices initially colored white
- Then colored grey when discovered
- Then black when finished

Depth-First Search: The Code

```
DFS(G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```


Depth-First Search: The Code

```
DFS(G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

What does `u->d` represent?

Depth-First Search: The Code

```
DFS(G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

What does `u->f` represent?

Depth-First Search: The Code

```
DFS(G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

Will all vertices eventually be colored black?

Depth-First Search: The Code

```
DFS(G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

What will be the running time?

Depth-First Search: The Code

```
DFS(G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

Running time: $O(n^2)$ because call DFS_Visit on each vertex, and the loop over Adj[] can run as many as $|V|$ times

Depth-First Search: The Code

```
DFS(G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

BUT, there is actually a tighter bound.

How many times will DFS_Visit() actually be called?

Depth-First Search: The Code

```
DFS(G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

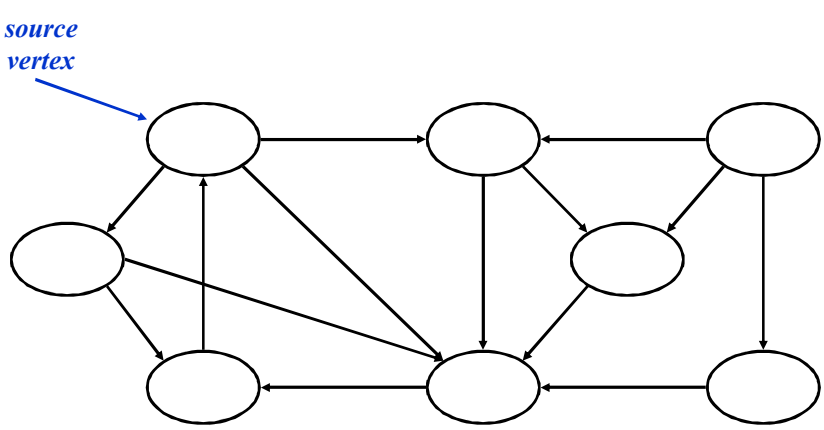
```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

So, running time of DFS = O(V+E)

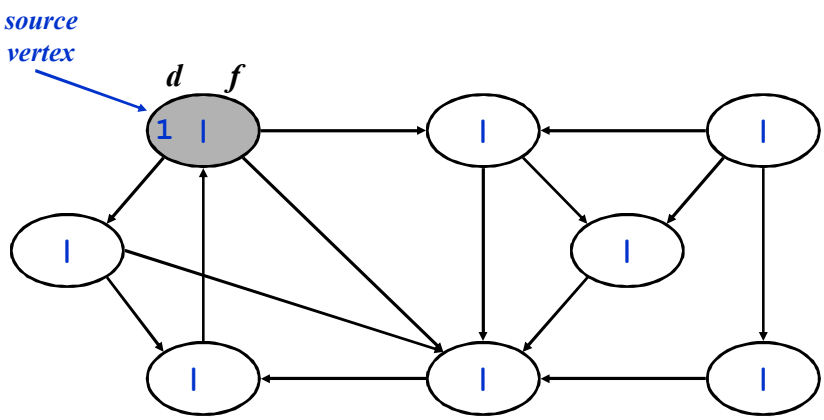
Depth-First Search Analysis

- This running time argument is an informal example of *amortized analysis*
  - "Charge" the exploration of edge to the edge:
    - Each loop in DFS_Visit can be attributed to an edge in the graph
    - Runs once/edge if directed graph, twice if undirected
    - Thus loop will run in $O(E)$ time, algorithm $O(V + E)$
  - Storage requirement is $O(V + E)$, since adjacent list requires $O(V + E)$ storage

DFS Example

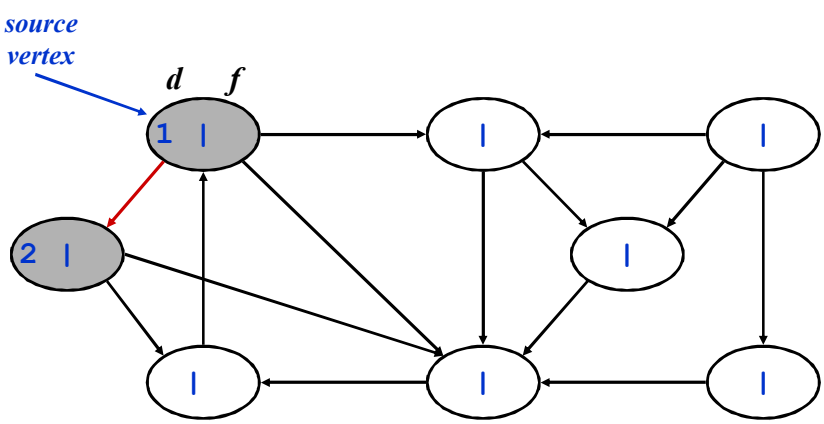


DFS Example



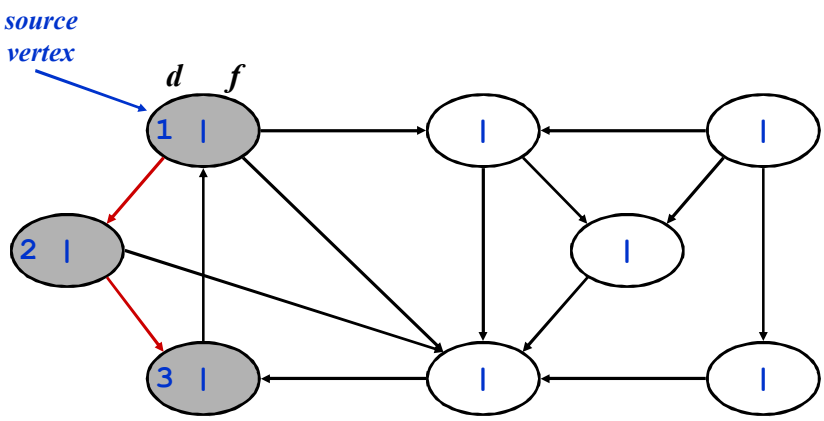
Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

DFS Example



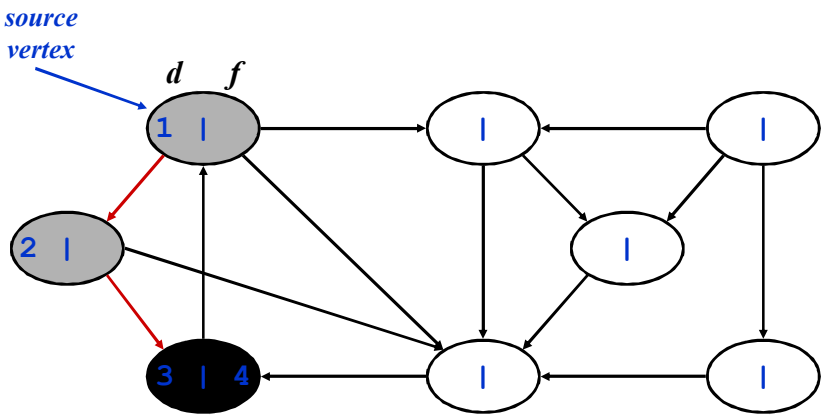
Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

DFS Example



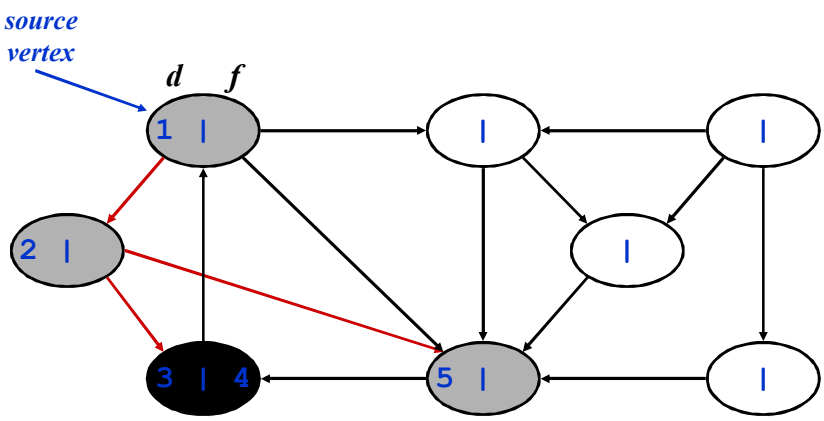
Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

DFS Example



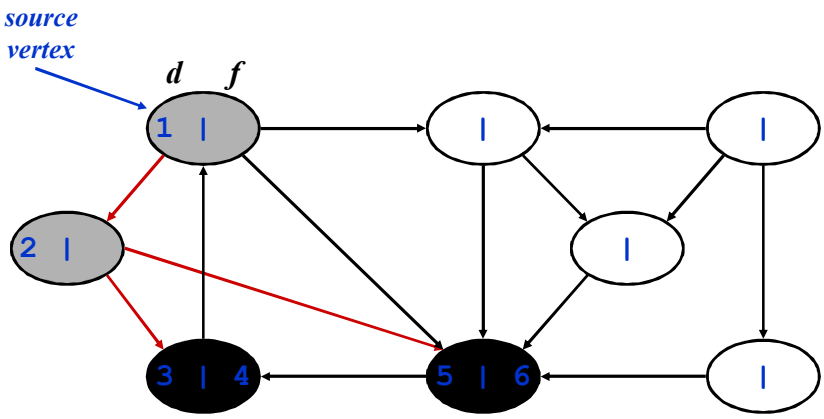
Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

DFS Example



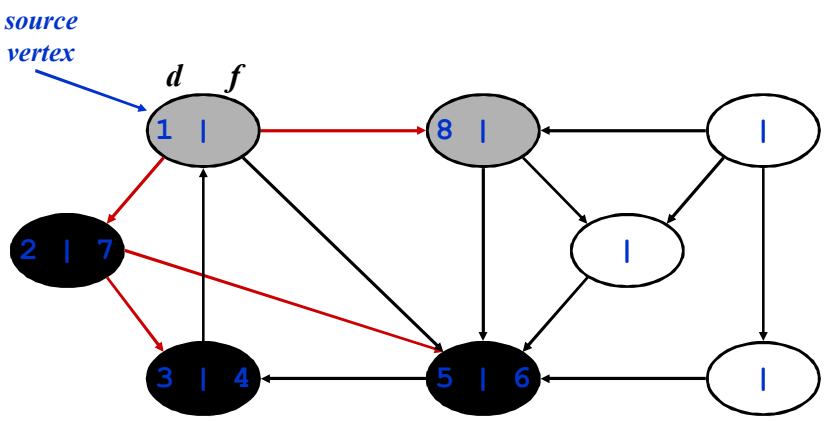
Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

DFS Example



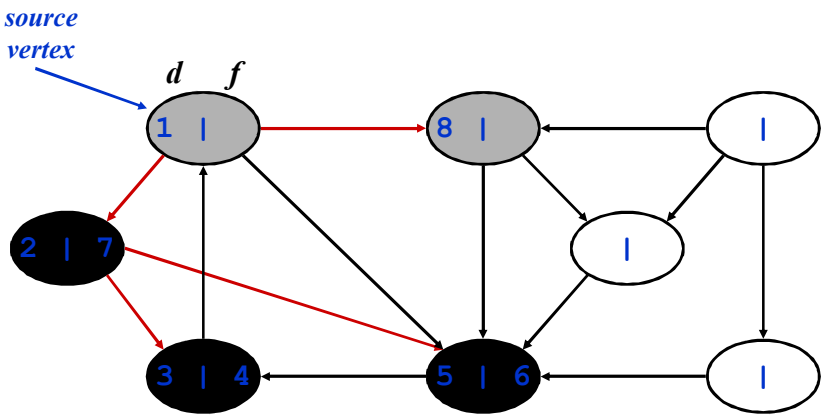
Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

DFS Example



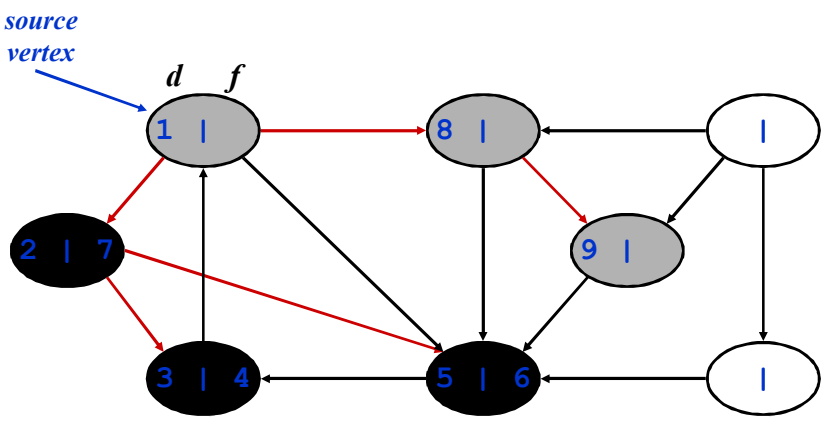
Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

DFS Example



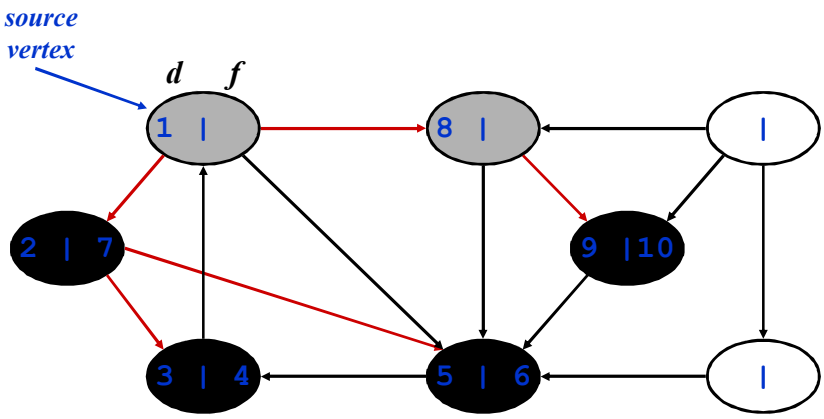
Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

DFS Example



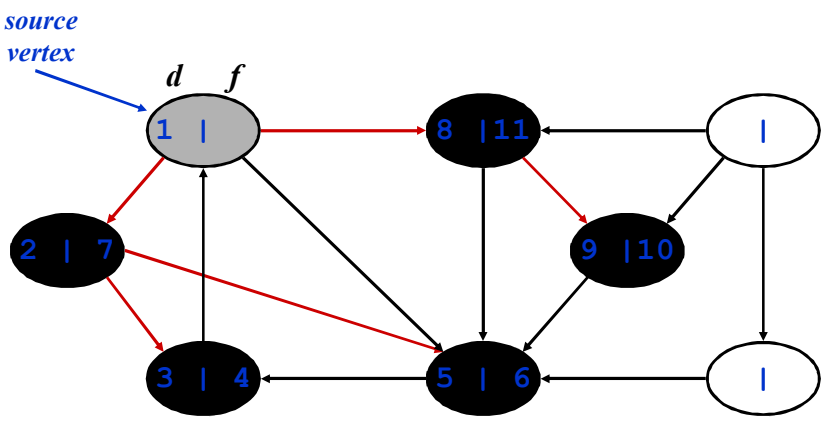
Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

DFS Example

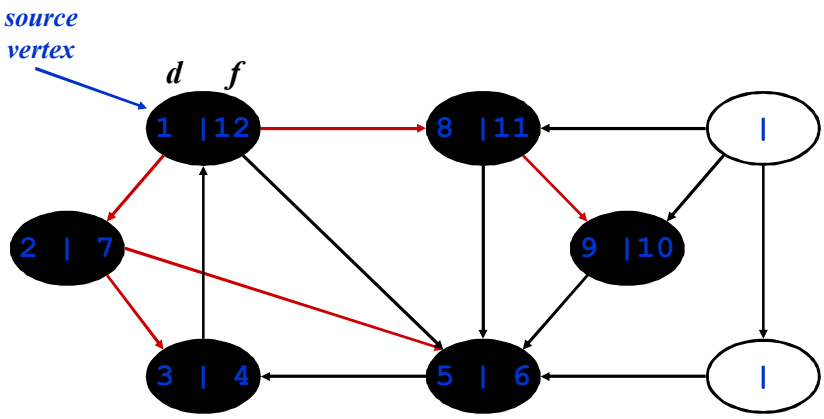


Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

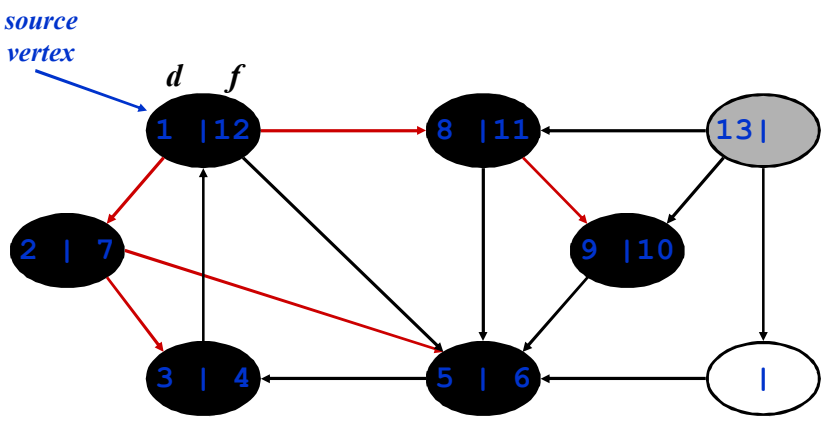
DFS Example



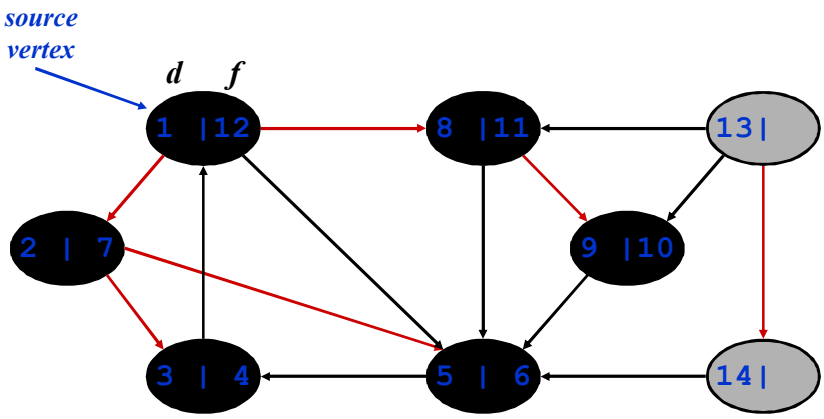
DFS Example



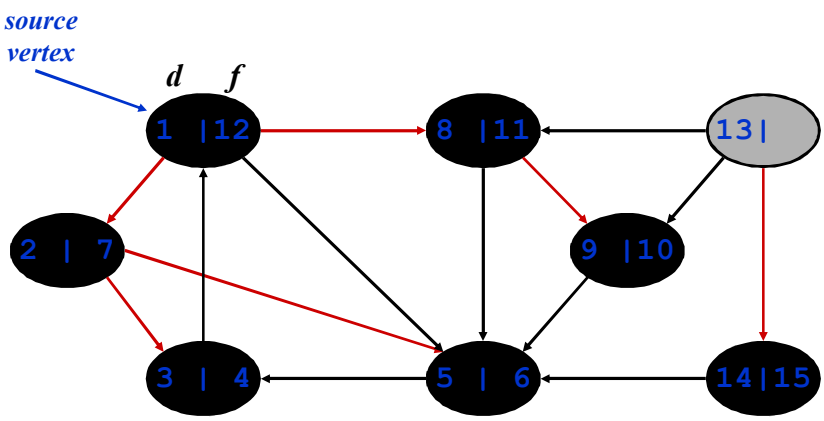
DFS Example



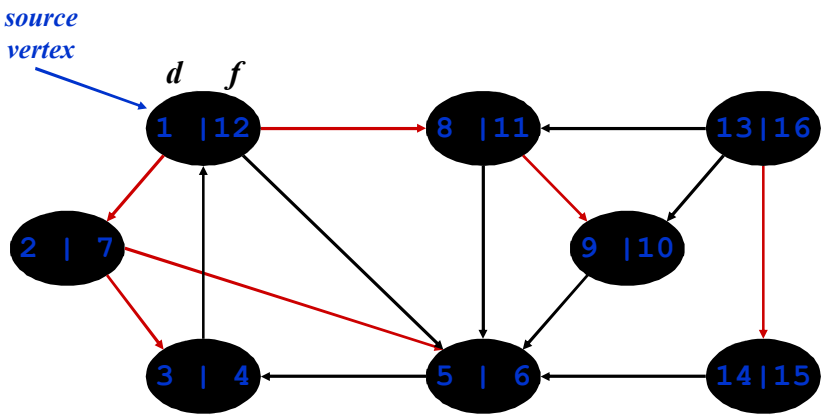
DFS Example



DFS Example



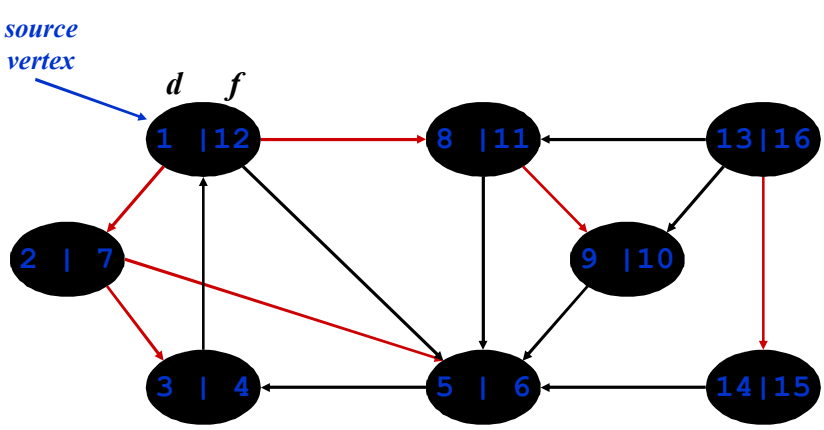
DFS Example



DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
  - *Tree edge*: encounter new (white) vertex
    - The tree edges form a spanning forest
    - *Can tree edges form cycles? Why or why not?*

DFS: Kinds of edges

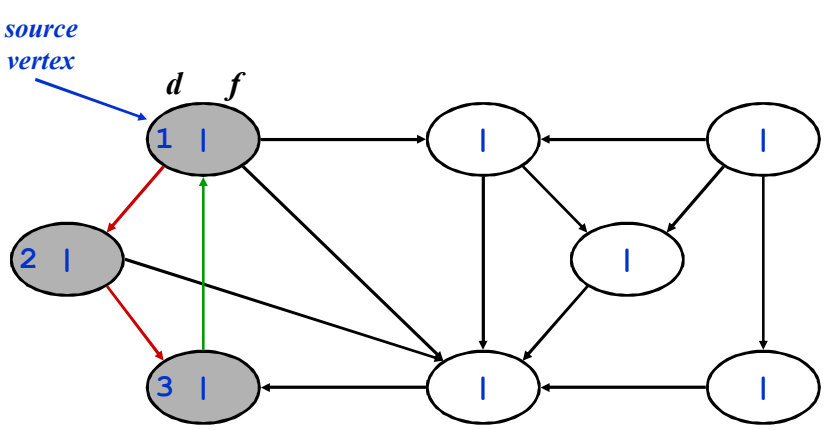


Tree edges

DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
  - *Tree edge*: encounter new (white) vertex
  - *Back edge*: from descendent to ancestor
    - Encounter a grey vertex (grey to grey)

DFS: Kinds of edges

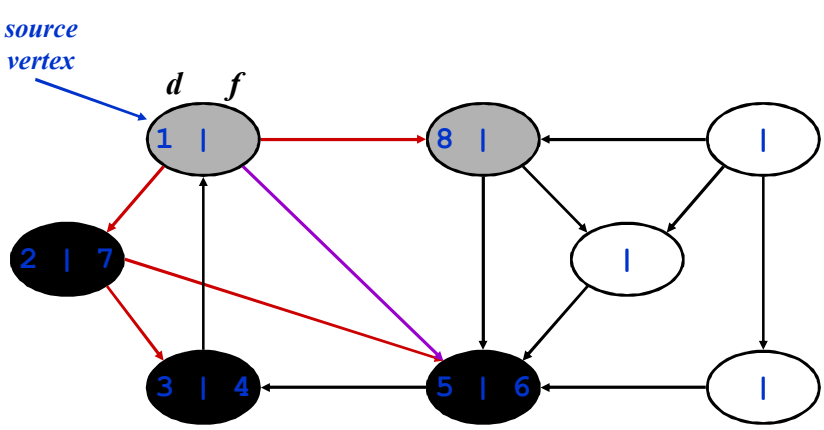


Tree edges *Back edges*

DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
  - *Tree edge*: encounter new (white) vertex
  - *Back edge*: from descendent to ancestor
  - *Forward edge*: from ancestor to descendent
    - Not a tree edge, though
    - From grey node to black node

DFS: Kinds of edges

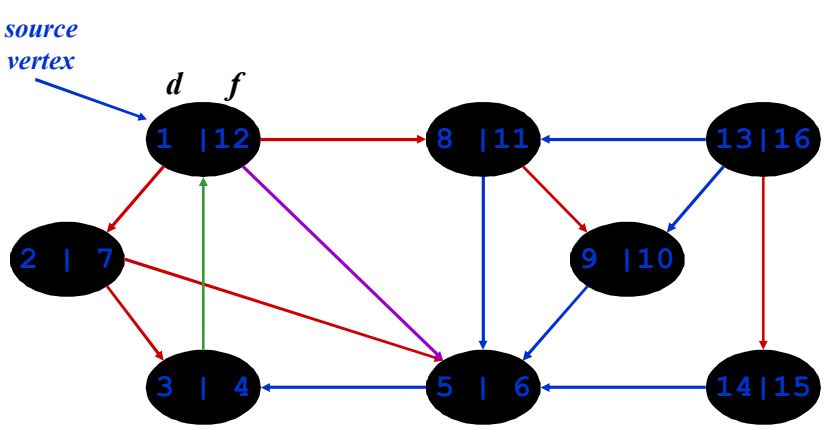


Tree edges *Back edges* *Forward edges*

DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
  - *Tree edge*: encounter new (white) vertex
  - *Back edge*: from descendent to ancestor
  - *Forward edge*: from ancestor to descendent
  - *Cross edge*: between a tree or subtrees
    - From a grey node to a black node

DFS: Kinds of edges

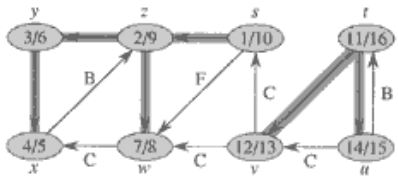


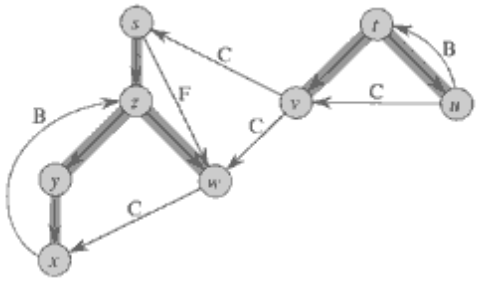
Tree edges *Back edges* *Forward edges* *Cross edges*

DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
  - *Tree edge*: encounter new (white) vertex
  - *Back edge*: from descendent to ancestor
  - *Forward edge*: from ancestor to descendent
  - *Cross edge*: between a tree or subtrees
- Note: tree and back edges are very important; some algorithms use forward and cross edges

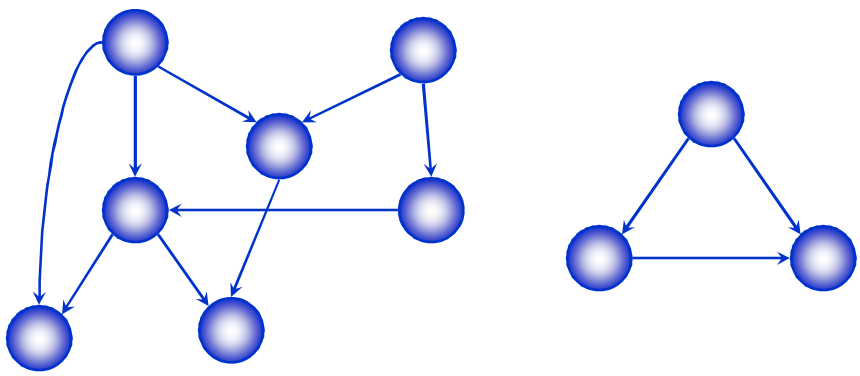
DFS: Kinds of edges





Directed Acyclic Graphs

- A *directed acyclic graph* or *DAG* is a directed graph with no directed cycles



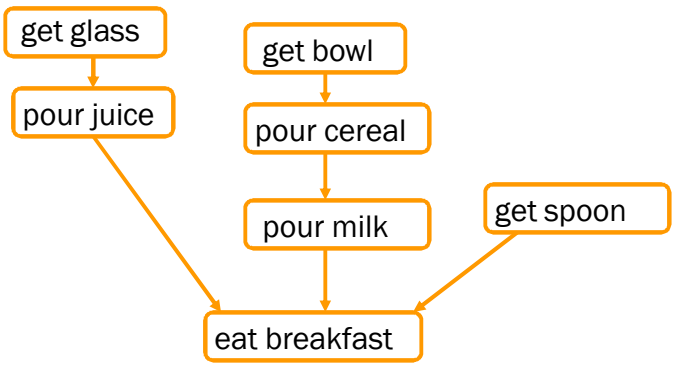
Topological Sort

- A *topological sort* of a DAG is
  - a linear ordering of all vertices of the graph G such that vertex u comes before vertex v if (u, v) is an edge in G.
- DAG indicates precedence among events:
  - events are graph vertices, edge from u to v means event u has precedence over event v
- Real-world example:
  - getting dressed
  - course registration
  - tasks for eating meal

Precedence Example

- Tasks that have to be done to eat breakfast:
  - get glass, pour juice, get bowl, pour cereal, pour milk, get spoon, eat.
- Certain events must happen in a certain order (ex: get bowl before pouring milk)
- For other events, it doesn't matter (ex: get bowl and get spoon)

Precedence Example



Order: glass, juice, bowl, cereal, milk, spoon, eat.

Why Acyclic?

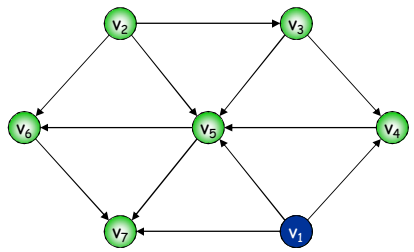
- Why must directed graph by acyclic for the topological sort problem?
- Otherwise, no way to order events linearly without violating a precedence constraint.

Topological Sort Algorithm

```
TOPOLOGICAL-SORT(G)
call DFS(G) to compute finishing times f[v] for each vertex v
as each vertex is finished, insert it onto the front of a linked list
return the linked list of vertices
```

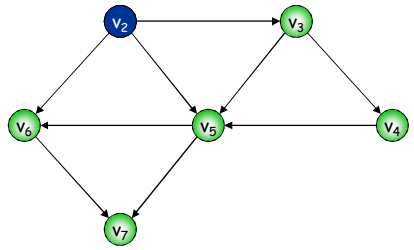
- Time: $O(V + E)$
- Correctness: Want to prove that
  $(u, v) \in E(G) \Rightarrow f(u) > f(v)$

Topological Sort: Using In-degree



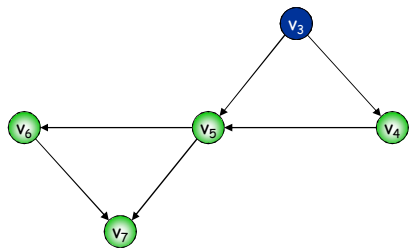
Topological order:

Topological Sort: Using In-degree



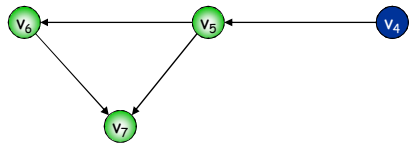
Topological order: v_1

Topological Sort: Using In-degree



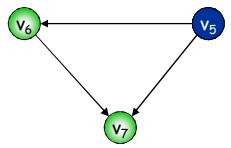
Topological order: v_1, v_2

Topological Sort: Using In-degree



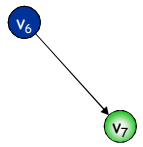
Topological order: v_1, v_2, v_3

Topological Sort: Using In-degree



Topological order: v_1, v_2, v_3, v_4

Topological Sort: Using In-degree

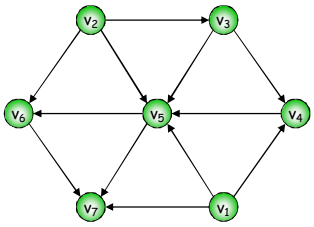


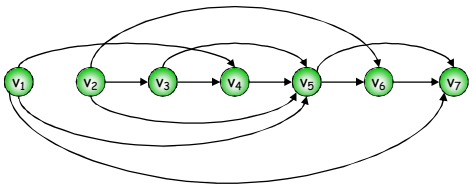
Topological order: v_1, v_2, v_3, v_4, v_5

Topological Sort: Using In-degree

Topological order: $v_1, v_2, v_3, v_4, v_5, v_6$

Topological Sort: Using In-degree





Topological order: $v_1, v_2, v_3, v_4, v_5, v_6, v_7$.

Topological Sort: Using In-degree

Steps for finding the topological ordering of a DAG:

Step-1: Compute in-degree for each of the vertices present in the DAG and initialize the count of visited nodes as 0;

Step-2: Add all **vertices with in-degree equals 0** into a queue

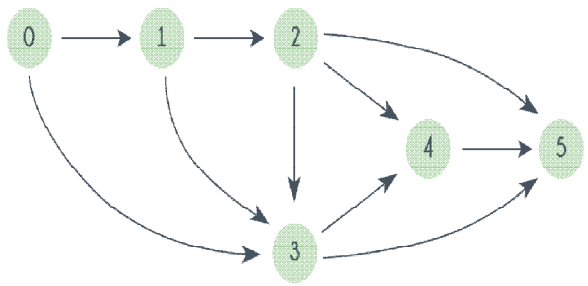
Step-3: Remove a vertex from the queue and then

- Increment count of visited nodes by 1;
- Decrease in-degree by 1 for all its neighboring nodes;
- If in-degree of a neighboring node is reduced to zero, then add it to the queue;

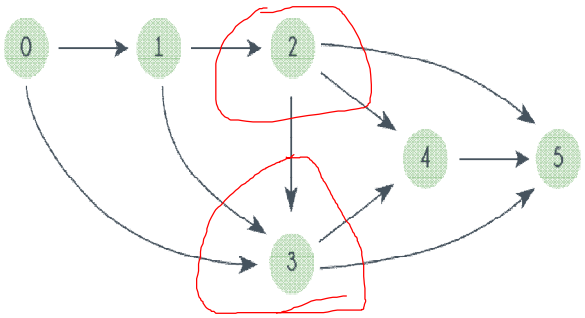
Step 4: Repeat Step 3 until **the queue is empty**;

Step 5: If count of visited nodes is **not** equal to the number of nodes in the graph then the topological sort is not possible for the given graph.

Topological Sort: Using In-degree



Topological Sort: Using In-degree

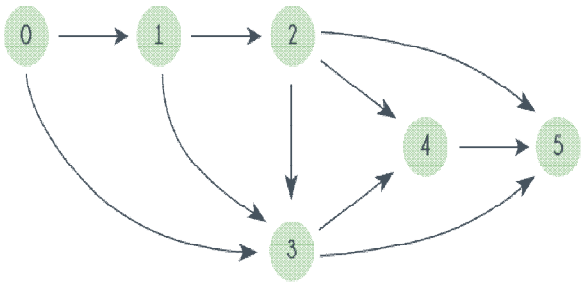


QUEUE : 0

	0	1	2	3	4	5
in_degree	0	1	2	2	2	3

T : --

Topological Sort: Using In-degree

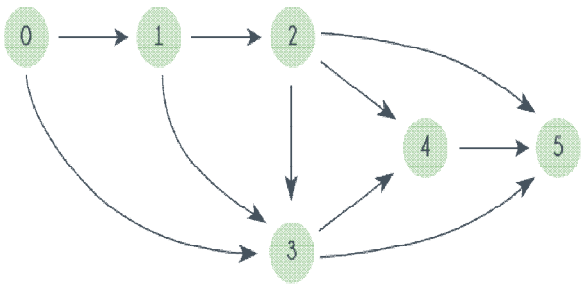


QUEUE : 1

	0	1	2	3	4	5
in_degree	0	0	1	1	2	3

T : 0

Topological Sort: Using In-degree

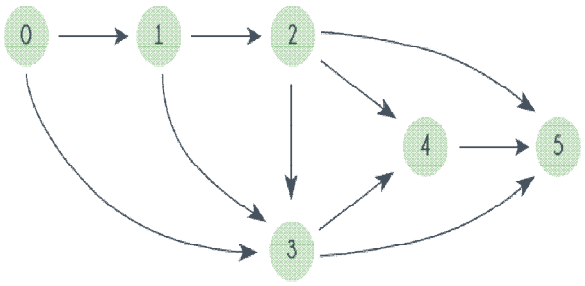


QUEUE : 2

	0	1	2	3	4	5
in_degree	0	0	0	1	1	3

T : 0 1

Topological Sort: Using In-degree

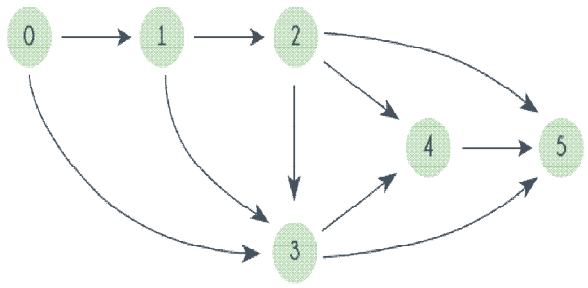


QUEUE : 3

	0	1	2	3	4	5
in_degree	0	0	0	0	1	2

T : 0 1 2

Topological Sort: Using In-degree

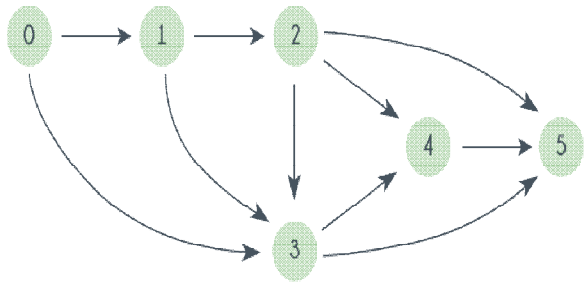


QUEUE : 4

	0	1	2	3	4	5
in_degree	0	0	0	0	0	1

T : 0 1 2 3

Topological Sort: Using In-degree

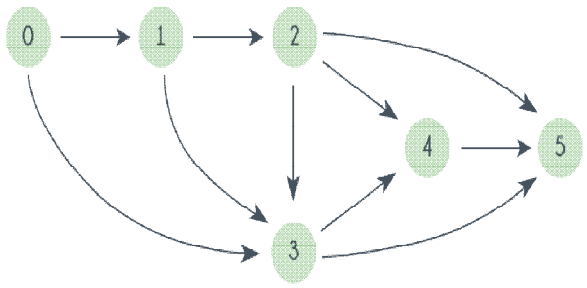


QUEUE : 5

	0	1	2	3	4	5
in_degree	0	0	0	0	0	0

T : 0 1 2 3 4

Topological Sort: Using In-degree



QUEUE : --

	0	1	2	3	4	5
in_degree	0	0	0	0	0	0

T : 0 1 2 3 4 5